Documento de Casos de Teste

ServeRest API — testes de API com Postman

1. Informações do projeto

Projeto / Sistema testado	ServeRest — API REST que simula uma loja virtual (e-commerce)
Versão / Ambiente	API versão 3.2.2 (conforme o Swagger) / https://serverest.dev (servidor público online)
Responsável (QA)	Yan Vieira
Início dos testes	21/10/2026
Término dos testes	21/10/2026
Última atualização	21/10/2026
Documentação de referência	Swagger da API: https://serverest.dev/swagger.json?lang=pt-BR
Escopo do teste	• Login: entrega do token, credenciais inválidas, campos ausentes e token inválido. • Usuários: listar, cadastrar, buscar por ID, editar e excluir (incluindo a regra do usuário com carrinho). • Produtos: listar, cadastrar, editar e excluir, com as regras de permissão de administrador e de nome único. • Carrinhos: cadastrar, regras de 1 carrinho por usuário e de estoque, concluir e cancelar compra.
Fora do escopo	• Testes de carga e de desempenho (a própria ServeRest pede cuidado com esse tipo de teste). • Segurança aprofundada (por exemplo, o fato de a lista de usuários mostrar a senha será apenas registrado como observação). • Filtros por parâmetros, edição com ID inexistente e buscas por ID de produto e de carrinho além dos usados nos casos. • Front-end da loja (front.serverest.dev) e testes automatizados (previstos em outro projeto do portfólio).
Dispositivos / Sistema operacional	Computador com Windows 10; Postman versão 12.31.0 (aplicativo para desktop).
Evidências	• RUN/00 - Preparação: PRINT 1 a PRINT 7 e PRINT 1 e 2 (GRID) — criação da massa de dados. • RUN/01 - Login: PRINT (GRID) e PRINT 1 a PRINT 4. • RUN/02 - Usuários: PRINT (GRID), PRINT 1 a PRINT 4 e PRINT RETESTE CT-USR-05. • RUN/03 - Produtos: PRINT (GRID) e PRINT 1 a PRINT 4. • RUN/04 - Carrinhos: PRINT (GRID) e PRINT 1 a PRINT 4.
Ferramentas utilizadas	• Postman: envio das requisições e conferência das respostas. • Swagger da ServeRest: consulta das regras e dos formatos de dados. • Captura de tela ou exportação dos resultados do Postman: evidências. • Microsoft Word: documentação. • GitHub: publicação do projeto.

2. Resumo dos casos de teste

Total: 32 casos. Passou: 30. Falhou: 2. Não executado: 0.

ID	Título	Módulo	Prioridade	Status	Bug
CT-LOG-01	Fazer login com um administrador válido	Login	Alta	Passou	—
CT-LOG-02	Fazer login com um usuário comum válido	Login	Alta	Passou	—
CT-LOG-03	Tentar fazer login com a senha incorreta	Login	Alta	Passou	—
CT-LOG-04	Tentar fazer login com um e-mail não cadastrado	Login	Alta	Passou	—
CT-LOG-05	Tentar fazer login sem informar o e-mail (teste exploratório)	Login	Média	Passou	—
CT-LOG-06	Tentar fazer login sem informar a senha (teste exploratório)	Login	Média	Passou	—
CT-LOG-07	Tentar fazer login com e-mail em formato inválido (teste exploratório)	Login	Baixa	Passou	—
CT-LOG-08	Usar um token inválido em uma rota protegida	Login – autenticação	Alta	Passou	—
CT-USR-01	Listar os usuários cadastrados	Usuários	Alta	Passou	—
CT-USR-02	Cadastrar um usuário com dados válidos	Usuários	Alta	Passou	—
CT-USR-03	Tentar cadastrar um usuário com e-mail já utilizado	Usuários	Alta	Passou	—
CT-USR-04	Buscar um usuário por um ID existente	Usuários	Média	Passou	—
CT-USR-05	Buscar um usuário por um ID que não existe	Usuários	Média	Passou	—
CT-USR-06	Editar os dados de um usuário existente	Usuários	Alta	Passou	—
CT-USR-07	Excluir um usuário que não tem carrinho	Usuários	Alta	Passou	—
CT-USR-08	Tentar excluir um usuário que tem carrinho	Usuários	Alta	Passou	—
CT-PRD-01	Listar os produtos cadastrados (sem token)	Produtos	Alta	Passou	—
CT-PRD-02	Cadastrar um produto como administrador	Produtos	Alta	Passou	—
CT-PRD-03	Tentar cadastrar um produto com nome já existente	Produtos	Alta	Passou	—

CT-PRD-04	Tentar cadastrar um produto sem token	Produtos	Alta	Passou	—
CT-PRD-05	Tentar cadastrar um produto com usuário que não é administrador	Produtos	Alta	Passou	—
CT-PRD-06	Editar um produto como administrador	Produtos	Média	Passou	—
CT-PRD-07	Excluir um produto que não está em nenhum carrinho	Produtos	Média	Passou	—
CT-PRD-08	Tentar excluir um produto que está dentro de um carrinho	Produtos	Alta	Falhou	BUG-001
CT-CAR-01	Cadastrar um carrinho com dados válidos	Carrinhos	Alta	Passou	—
CT-CAR-02	Reduzir o estoque do produto ao cadastrar o carrinho	Carrinhos	Alta	Passou	—
CT-CAR-03	Tentar cadastrar um carrinho sem token	Carrinhos	Alta	Passou	—
CT-CAR-04	Tentar cadastrar um segundo carrinho para o mesmo usuário	Carrinhos	Alta	Passou	—
CT-CAR-05	Tentar cadastrar um carrinho com um produto que não existe	Carrinhos	Média	Passou	—
CT-CAR-06	Tentar cadastrar um carrinho com quantidade maior que o estoque	Carrinhos	Alta	Passou	—
CT-CAR-07	Concluir a compra e excluir o carrinho	Carrinhos – concluir compra	Alta	Passou	—
CT-CAR-08	Cancelar a compra e devolver os produtos ao estoque	Carrinhos – cancelar compra	Alta	Falhou	BUG-002

3. Legenda

Prioridade do caso de teste

- Alta: se falhar, a API não cumpre uma função principal.
- Média: função importante, mas existe um jeito de contornar.
- Baixa: detalhe ou fluxo pouco usado.

Status do caso

- Passou: o resultado obtido foi igual ao esperado.
- Falhou: o resultado obtido foi diferente do esperado (gera um bug).
- Bloqueado: não foi possível executar por causa de outro problema.
- Não executado: caso escrito, ainda não executado.

Tipos de teste

- Funcional – Positivo: o caminho normal, com dados corretos.
- Funcional – Negativo: o que a API não deve permitir.
- Validação de campos: dados ausentes ou em formato inválido.
- Autenticação e permissão: acesso sem token, com token inválido ou sem ser administrador.
- Regra de negócio: uma regra específica do e-commerce (estoque, carrinho único, exclusões bloqueadas).
- Exploratório: o Swagger não diz o que deve acontecer; o QA define o esperado pela regra e anota o que a API realmente fizer.

Severidade dos bugs

- Crítica: a API para ou perde dados.
- Alta: uma função principal não funciona e não há contorno.
- Média: a função falha, mas há contorno parcial.
- Baixa: problema de mensagem ou de documentação, sem impacto nos dados.

Códigos de status que aparecem nos testes

Código	Nome	Significado simples
200	OK	A API fez o que foi pedido.
201	Created	Um novo registro foi criado.
400	Bad Request	O pedido está errado ou fere uma regra (dado repetido, campo faltando, item inexistente).
401	Unauthorized	Falta o token, ele é inválido, expirou, ou o login está errado.
403	Forbidden	O usuário está logado, mas não tem permissão (não é administrador).

4. Massa de dados e preparação do ambiente

A ServeRest é um servidor público usado por muita gente. Por isso, os e-mails e os nomes de produto precisam ser únicos. Nos dados abaixo, o <código> é um número gerado a cada execução (o Postman gera sozinho; no teste manual, escolha um número que ainda não foi usado).

Usuários

Usuário	E-mail	Senha	Admin	Usado em
Usuário Admin	admin.qa.<código>@qa.com.br	teste123	true	Casos de Produtos e Login
Usuário Comum 1	comum1.qa.<código>@qa.com.br	teste123	false	LOG-02, USR-01 a 06, PRD-05, CAR-01
Usuário Comum 2	comum2.qa.<código>@qa.com.br	teste123	false	CAR-02
Usuário Comum 3	comum3.qa.<código>@qa.com.br	teste123	false	CAR-04
Usuário Comum 4	comum4.qa.<código>@qa.com.br	teste123	false	CAR-05, CAR-06
Usuário Comum 5	comum5.qa.<código>@qa.com.br	teste123	false	CAR-07
Usuário Comum 6	comum6.qa.<código>@qa.com.br	teste123	false	CAR-08
Usuário Comum 7	comum7.qa.<código>@qa.com.br	teste123	false	USR-08 (já tem carrinho)

Produtos

Produto	Nome	Preço	Descrição	Estoque	Usado em
Produto A	Teclado QA <código>	250	Teclado mecânico	100	PRD-01, PRD-03, CAR-01, CAR-03, CAR-04, CAR-08
Produto B	Mouse QA <código>	120	Mouse sem fio	50	CAR-02, CAR-04, CAR-06, CAR-07
Produto C	Monitor QA <código>	900	Monitor 24 polegadas	10	PRD-08 (já está no carrinho do Comum 7)

Preparação do ambiente (fazer uma vez, antes dos casos)

- Cadastrar o Usuário Admin (administrador “true”) e os Usuários Comuns 1 a 7 (administrador “false”), com POST /usuarios. Anotar o _id de cada um.
- Fazer login de cada usuário com POST /login e guardar o token de cada um. Cada token vale 10 minutos; se aparecer 401 sem motivo, refazer o login.
- Com o token do Admin, cadastrar os Produtos A, B e C com POST /produtos. Anotar o _id de cada um.
- Com o token do Usuário Comum 7, criar um carrinho com 1 unidade do Produto C (POST /carrinhos).

Os casos que criam um usuário ou um produto só para serem excluídos ou editados (CT-USR-02, CT-USR-07, CT-PRD-02, CT-PRD-06 e CT-PRD-07) cadastram esses dados dentro do próprio caso.

5. Regras de negócio

As regras abaixo vêm da documentação Swagger da ServeRest (fonte “Swagger”) foram definidas pelo QA quando a documentação não diz o que deve acontecer (fonte “QA”) ou foram descobertas durante a execução dos testes (fonte “Execução”). Elas são a base dos resultados esperados dos casos.

ID	Regra	Fonte
RN-01	O login com e-mail e senha cadastrados devolve a mensagem “Login realizado com sucesso” e um token (campo authorization) que vale 600 segundos (10 minutos).	Swagger
RN-02	O login com e-mail ou senha inválidos devolve status 401 e a mensagem “Email e/ou senha inválidos”, sem token.	Swagger
RN-03	O login exige e-mail (em formato válido) e senha preenchidos. Se faltar algum, ou se o e-mail for inválido, a API recusa o pedido com status 400 e não entrega token.	QA
RN-04	As rotas protegidas exigem um token válido no cabeçalho Authorization. Token ausente, inválido ou expirado devolve status 401.	Swagger
RN-05	O cadastro de usuário usa nome, email, password e administrador (“true” ou “false”). Não é permitido cadastrar um e-mail já usado (status 400).	Swagger
RN-06	A lista de usuários mostra quantidade e usuarios. Buscar um usuário por um ID que não existe devolve status 400 e “Usuário não encontrado”.	Swagger
RN-07	A edição de usuário (PUT) altera os dados e devolve “Registro alterado com sucesso”. O e-mail de outro usuário não pode ser usado.	Swagger
RN-08	A exclusão de usuário é permitida sem carrinho. Usuário com carrinho não pode ser excluído (status 400).	Swagger
RN-09	Listar produtos é público. Cadastrar, editar e excluir produtos exige token de administrador: sem token devolve 401; com usuário comum devolve 403.	Swagger
RN-10	Não é permitido cadastrar dois produtos com o mesmo nome (status 400).	Swagger
RN-11	Um produto que faz parte de um carrinho não pode ser excluído (status 400).	Swagger
RN-12	Cada usuário pode ter apenas 1 carrinho, ligado ao usuário do token. O carrinho não aceita produto inexistente, produto repetido nem quantidade maior que o estoque (status 400).	Swagger
RN-13	Criar um carrinho reduz o estoque dos produtos. Cancelar a compra exclui o carrinho e devolve o estoque. Concluir a compra exclui o carrinho e não devolve o estoque.	Swagger
RN-14	Nas rotas que usam ID na URL, o ID precisa ter exatamente 16 caracteres alfanuméricos; fora desse formato a API responde 400 com {"id": "id deve ter exatamente 16 caracteres alfanuméricos"}. Descoberta na execução; não está descrita no Swagger.	Execução

6. Casos de teste

Grupo A — Login

Caso de teste CT-LOG-01	
ID	CT-LOG-01
Título	**Fazer login com um administrador válido**
Módulo / Funcionalidade	Login
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-01
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Admin já está cadastrado.
Dados de teste	Usuário Admin: e-mail e senha da seção 4.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /login com o corpo abaixo, usando o e-mail e a senha do Usuário Admin. `{ "email": "<e-mail do Admin>", "password": "teste123" }`	O corpo está em formato JSON e tem os campos email e password.
2	Enviar a requisição.	A API responde com status 200 (OK).
3	Ler o campo message da resposta.	Mostra “Login realizado com sucesso”.
4	Ler o campo authorization da resposta.	O campo existe e começa com “Bearer ”, seguido de um texto longo (o token).
5	Guardar o token do Admin.	O token fica guardado para os casos de Produtos (ele vale por 10 minutos).

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-LOG-02	
ID	CT-LOG-02
Título	**Fazer login com um usuário comum válido**
Módulo / Funcionalidade	Login
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-01
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 1 já está cadastrado.
Dados de teste	Usuário Comum 1: e-mail e senha da seção 4.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /login com o corpo abaixo, usando o e-mail e a senha do Usuário Comum 1. `{ "email": "<e-mail do Comum 1>", "password": "teste123" }`	O corpo está em formato JSON e tem os campos email e password.
2	Enviar a requisição.	A API responde com status 200 (OK).
3	Ler o campo message da resposta.	Mostra “Login realizado com sucesso”.
4	Ler o campo authorization da resposta.	O campo existe e começa com “Bearer ”, seguido do token.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-LOG-03	
ID	CT-LOG-03
Título	**Tentar fazer login com a senha incorreta**
Módulo / Funcionalidade	Login
Tipo de teste	Funcional – Negativo
Prioridade	Alta
Regra de negócio	RN-02
Pré-condições	• Preparação do ambiente feita (seção 4).
Dados de teste	E-mail do Usuário Admin (cadastrado) e a senha errada “senhaErrada999”.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /login com o e-mail do Admin e uma senha errada. `{ "email": "<e-mail do Admin>", "password": "senhaErrada999" }`	O corpo está em formato JSON.
2	Enviar a requisição.	A API responde com status 401 (Unauthorized).
3	Ler o campo message da resposta.	Mostra “Email e/ou senha inválidos”.
4	Procurar o campo authorization na resposta.	O campo não existe: nenhum token é entregue.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 1 e PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-LOG-04	
ID	CT-LOG-04
Título	**Tentar fazer login com um e-mail não cadastrado**
Módulo / Funcionalidade	Login
Tipo de teste	Funcional – Negativo
Prioridade	Alta
Regra de negócio	RN-02
Pré-condições	• Nenhuma.
Dados de teste	E-mail “naoexiste.qa@qa.com.br” (nunca cadastrado) e senha “teste123”.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /login com um e-mail que não existe no sistema. `{ "email": "naoexiste.qa@qa.com.br", "password": "teste123" }`	O corpo está em formato JSON.
2	Enviar a requisição.	A API responde com status 401 (Unauthorized).
3	Ler o campo message da resposta.	Mostra “Email e/ou senha inválidos”.
4	Procurar o campo authorization na resposta.	O campo não existe: nenhum token é entregue.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-LOG-05	
ID	CT-LOG-05
Título	**Tentar fazer login sem informar o e-mail (teste exploratório)**
Módulo / Funcionalidade	Login
Tipo de teste	Validação de campos (exploratório)
Prioridade	Média
Regra de negócio	RN-03
Pré-condições	• Nenhuma. • O Swagger não descreve este caso. O esperado abaixo vem da regra RN-03, definida pelo QA.
Dados de teste	Apenas a senha “teste123”; o campo email não é enviado.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /login com o corpo abaixo, sem o campo email. `{ "password": "teste123" }`	O corpo tem só o campo password.
2	Enviar a requisição.	A API recusa o pedido com status 400 (Bad Request). Se vier outro status, anotar em Observações.
3	Ler o campo message (ou os campos de erro) da resposta.	A mensagem indica que o e-mail é obrigatório. Anotar o texto exato em Observações.
4	Procurar o campo authorization na resposta.	O campo não existe: nenhum token é entregue.

Resultado obtido	Conforme esperado: status 400 e nenhum token. Resposta da API: {"email": "email é obrigatório"}.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 2 e PRINT 3. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	O Swagger do /login só documenta 200 e 401; o 400 com mensagem por campo não está documentado.
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-LOG-06	
ID	CT-LOG-06
Título	**Tentar fazer login sem informar a senha (teste exploratório)**
Módulo / Funcionalidade	Login
Tipo de teste	Validação de campos (exploratório)
Prioridade	Média
Regra de negócio	RN-03
Pré-condições	• Preparação do ambiente feita (seção 4). • O Swagger não descreve este caso. O esperado abaixo vem da regra RN-03, definida pelo QA.
Dados de teste	Apenas o e-mail do Usuário Admin; o campo password não é enviado.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /login com o corpo abaixo, sem o campo password. `{ "email": "<e-mail do Admin>" }`	O corpo tem só o campo email.
2	Enviar a requisição.	A API recusa o pedido com status 400 (Bad Request). Se vier outro status, anotar em Observações.
3	Ler o campo message (ou os campos de erro) da resposta.	A mensagem indica que a senha é obrigatória. Anotar o texto exato em Observações.
4	Procurar o campo authorization na resposta.	O campo não existe: nenhum token é entregue.

Resultado obtido	Conforme esperado: status 400 e nenhum token. Resposta da API: {"password": "password é obrigatório"}.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 3. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	O Swagger do /login só documenta 200 e 401; o 400 com mensagem por campo não está documentado.
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-LOG-07	
ID	CT-LOG-07
Título	**Tentar fazer login com e-mail em formato inválido (teste exploratório)**
Módulo / Funcionalidade	Login
Tipo de teste	Validação de campos (exploratório)
Prioridade	Baixa
Regra de negócio	RN-03
Pré-condições	• Nenhuma. • O Swagger não descreve este caso. O esperado abaixo vem da regra RN-03, definida pelo QA.
Dados de teste	E-mail “email-sem-arroba” (sem @ e sem domínio) e senha “teste123”.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /login com um e-mail que não tem o formato correto. `{ "email": "email-sem-arroba", "password": "teste123" }`	O corpo está em formato JSON.
2	Enviar a requisição.	A API recusa o pedido com status 400 (Bad Request). Se vier 401, anotar em Observações.
3	Ler o campo message da resposta.	A mensagem indica que o e-mail é inválido. Anotar o texto exato em Observações.
4	Procurar o campo authorization na resposta.	O campo não existe: nenhum token é entregue.

Resultado obtido	Conforme esperado: status 400 e nenhum token. Resposta da API: {"email": "email deve ser um email válido"}.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 3 e PRINT 4. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	O Swagger do /login só documenta 200 e 401; o 400 com mensagem por campo não está documentado.
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-LOG-08	
ID	CT-LOG-08
Título	**Usar um token inválido em uma rota protegida**
Módulo / Funcionalidade	Login – autenticação
Tipo de teste	Autenticação e permissão
Prioridade	Alta
Regra de negócio	RN-04
Pré-condições	• Preparação do ambiente feita (seção 4).
Dados de teste	Token inventado “Bearer abc.def.ghi”. Produto: nome “Produto Token Inválido QA”, preço 100, descrição “Teste”, quantidade 5.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /produtos, com o cabeçalho Authorization igual a “Bearer abc.def.ghi” e o corpo abaixo. `{ "nome": "Produto Token Inválido QA", "preco": 100, "descricao": "Teste", "quantidade": 5 }`	O cabeçalho e o corpo estão preenchidos.
2	Enviar a requisição.	A API responde com status 401 (Unauthorized).
3	Ler o campo message da resposta.	Mostra “Token de acesso ausente, inválido, expirado ou usuário do token não existe mais”.
4	Enviar GET /produtos?nome=Produto Token Inválido QA.	A resposta mostra quantidade 0: o produto não foi criado.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/01 - Login: PRINT 4. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Grupo B — Usuários

Caso de teste CT-USR-01	
ID	CT-USR-01
Título	**Listar os usuários cadastrados**
Módulo / Funcionalidade	Usuários
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-06
Pré-condições	• Preparação do ambiente feita (seção 4).
Dados de teste	Nenhum corpo. Para o filtro, usar o e-mail do Usuário Comum 1.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /usuarios, sem corpo e sem token.	A API responde com status 200 (OK).
2	Olhar a estrutura da resposta.	Existem o campo quantidade (um número) e o campo usuarios (uma lista).
3	Abrir um dos itens da lista.	O usuário tem os campos nome, email, password, administrador e _id.
4	Comparar o valor de quantidade com o número de itens da lista.	Os dois valores são iguais.
5	Enviar GET /usuarios?email=<e-mail do Comum 1>.	Status 200, quantidade igual a 1 e o único usuário da lista é o Comum 1.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-USR-02	
ID	CT-USR-02
Título	**Cadastrar um usuário com dados válidos**
Módulo / Funcionalidade	Usuários
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-05
Pré-condições	• Nenhuma.
Dados de teste	Novo usuário: nome “QA Novo Usuário”, e-mail “novo.qa.<código>@qa.com.br” (o <código> é um número que ainda não foi usado), senha “teste123”, administrador “false”.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /usuarios com o corpo abaixo. `{ "nome": "QA Novo Usuário", "email": "novo.qa.<código>@qa.com.br", "password": "teste123", "administrador": "false" }`	O corpo está em formato JSON, com os quatro campos.
2	Enviar a requisição.	A API responde com status 201 (Created).
3	Ler a resposta.	O campo message mostra “Cadastro realizado com sucesso” e existe um campo _id.
4	Enviar GET /usuarios/<_id recebido>.	Status 200.
5	Comparar os dados da resposta com os dados enviados.	Nome, e-mail, senha e administrador são iguais aos enviados.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-USR-03	
ID	CT-USR-03
Título	**Tentar cadastrar um usuário com e-mail já utilizado**
Módulo / Funcionalidade	Usuários
Tipo de teste	Funcional – Negativo
Prioridade	Alta
Regra de negócio	RN-05
Pré-condições	• Preparação do ambiente feita (seção 4): o e-mail do Usuário Comum 1 já existe.
Dados de teste	Nome “QA Repetido” com o mesmo e-mail do Usuário Comum 1, senha “teste123”, administrador “false”.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /usuarios usando o e-mail que já pertence ao Usuário Comum 1. `{ "nome": "QA Repetido", "email": "<e-mail do Comum 1>", "password": "teste123", "administrador": "false" }`	O corpo está em formato JSON.
2	Enviar a requisição.	A API responde com status 400 (Bad Request).
3	Ler o campo message da resposta.	Mostra “Este email já está sendo usado”.
4	Enviar GET /usuarios?email=<e-mail do Comum 1>.	A resposta mostra quantidade 1: nenhum usuário duplicado foi criado.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-USR-04	
ID	CT-USR-04
Título	**Buscar um usuário por um ID existente**
Módulo / Funcionalidade	Usuários
Tipo de teste	Funcional – Positivo
Prioridade	Média
Regra de negócio	RN-06
Pré-condições	• Preparação do ambiente feita (seção 4).
Dados de teste	_id do Usuário Comum 1 (anotado na preparação).

Passos de execução

Passo	Ação	Resultado esperado
1	Anotar o _id do Usuário Comum 1.	Você tem o _id em mãos.
2	Enviar GET /usuarios/<_id do Comum 1>.	A API responde com status 200 (OK).
3	Conferir os campos da resposta.	Aparecem nome, email, password, administrador (“false”) e _id.
4	Comparar o _id da resposta com o _id que você usou na busca.	Os dois são iguais.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-USR-05	
ID	CT-USR-05
Título	**Buscar um usuário por um ID que não existe**
Módulo / Funcionalidade	Usuários
Tipo de teste	Funcional – Negativo
Prioridade	Média
Regra de negócio	RN-06
Pré-condições	• Nenhuma.
Dados de teste	ID inventado com 16 caracteres (formato correto), mas que não existe no sistema: “naoExiste1234567”.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /usuarios/naoExiste1234567.	A API responde com status 400 (Bad Request).
2	Ler o campo message da resposta.	Mostra “Usuário não encontrado”.
3	Conferir o restante da resposta.	Não aparece nenhum dado de usuário (nome, e-mail, senha).

Resultado obtido	1ª execução: falhou no teste da mensagem. A API respondeu 400 com {"id": "id deve ter exatamente 16 caracteres alfanuméricos"}, porque o ID de teste tinha 17 caracteres (erro no dado de teste, não da API). Dado corrigido para “naoExiste1234567” (16 caracteres). Reteste: conforme esperado (400 e “Usuário não encontrado”).
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 2; PRINT 3 (detalhe da falha na 1ª execução); PRINT RETESTE CT-USR-05 (reteste passando). Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	Descoberta a regra RN-14 (ID com 16 caracteres), que não está no Swagger. Evidência do reteste: RUN/02 - Usuários/PRINT RETESTE CT-USR-05 RUN 02 - Usuários.png.
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-USR-06	
ID	CT-USR-06
Título	**Editar os dados de um usuário existente**
Módulo / Funcionalidade	Usuários
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-07
Pré-condições	• Preparação do ambiente feita (seção 4).
Dados de teste	Usuário Comum 1: novo nome “QA Comum 1 Editado”. O e-mail, a senha e o campo administrador continuam iguais.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /usuarios/<_id do Comum 1> e anotar o nome atual.	Status 200 e o nome atual está anotado.
2	Montar uma requisição PUT para /usuarios/<_id do Comum 1> com o corpo abaixo (e-mail, senha e administrador iguais aos de antes). `{ "nome": "QA Comum 1 Editado", "email": "<e-mail do Comum 1>", "password": "teste123", "administrador": "false" }`	O corpo tem os quatro campos.
3	Enviar a requisição.	A API responde com status 200 (OK).
4	Ler o campo message da resposta.	Mostra “Registro alterado com sucesso”.
5	Enviar GET /usuarios/<_id do Comum 1> de novo.	O nome agora é “QA Comum 1 Editado”; e-mail, senha e administrador não mudaram.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 3. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-USR-07	
ID	CT-USR-07
Título	**Excluir um usuário que não tem carrinho**
Módulo / Funcionalidade	Usuários
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-08
Pré-condições	• Nenhuma. O próprio caso cadastra o usuário que será excluído.
Dados de teste	Usuário descartável: nome “QA Descartável”, e-mail “descartavel.qa.<código>@qa.com.br”, senha “teste123”, administrador “false”.

Passos de execução

Passo	Ação	Resultado esperado
1	Cadastrar o usuário descartável com POST /usuarios e o corpo abaixo. Anotar o _id recebido. `{ "nome": "QA Descartável", "email": "descartavel.qa.<código>@qa.com.br", "password": "teste123", "administrador": "false" }`	Status 201 e o _id do usuário foi anotado.
2	Enviar DELETE /usuarios/<_id do descartável>.	A API responde com status 200 (OK).
3	Ler o campo message da resposta.	Mostra “Registro excluído com sucesso”.
4	Enviar GET /usuarios/<_id do descartável>.	Status 400 e a mensagem “Usuário não encontrado”: o usuário não existe mais.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 3 e PRINT 4. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-USR-08	
ID	CT-USR-08
Título	**Tentar excluir um usuário que tem carrinho**
Módulo / Funcionalidade	Usuários
Tipo de teste	Regra de negócio
Prioridade	Alta
Regra de negócio	RN-08
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 7 já tem um carrinho.
Dados de teste	_id do Usuário Comum 7.

Passos de execução

Passo	Ação	Resultado esperado
1	Anotar o _id do Usuário Comum 7.	Você tem o _id em mãos.
2	Enviar DELETE /usuarios/<_id do Comum 7>.	A API responde com status 400 (Bad Request).
3	Ler a resposta.	O campo message mostra “Não é permitido excluir usuário com carrinho cadastrado” e vem também o campo idCarrinho.
4	Enviar GET /usuarios/<_id do Comum 7>.	Status 200: o usuário continua cadastrado.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/02 - Usuários: PRINT 4. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Grupo C — Produtos

Caso de teste CT-PRD-01	
ID	CT-PRD-01
Título	**Listar os produtos cadastrados (sem token)**
Módulo / Funcionalidade	Produtos
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-09
Pré-condições	• Preparação do ambiente feita (seção 4): o Produto A já está cadastrado.
Dados de teste	Nenhum corpo e nenhum token. Para o filtro, usar o nome do Produto A.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /produtos sem o cabeçalho Authorization.	A API responde com status 200 (OK): listar produtos é público.
2	Olhar a estrutura da resposta.	Existem o campo quantidade (um número) e o campo produtos (uma lista).
3	Abrir um dos itens da lista.	O produto tem os campos nome, preco, descricao, quantidade e _id.
4	Enviar GET /produtos?nome=<nome do Produto A>.	Status 200, quantidade igual a 1 e o produto da lista é o Produto A.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-PRD-02	
ID	CT-PRD-02
Título	**Cadastrar um produto como administrador**
Módulo / Funcionalidade	Produtos
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-09
Pré-condições	• Preparação do ambiente feita (seção 4). • Token do Usuário Admin válido (ele vale 10 minutos; se der 401, refazer o login).
Dados de teste	Produto novo: nome “Headset QA <código>”, preço 300, descrição “Headset com microfone”, quantidade 20.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /produtos com o cabeçalho Authorization (token do Admin) e o corpo abaixo. `{ "nome": "Headset QA <código>", "preco": 300, "descricao": "Headset com microfone", "quantidade": 20 }`	O cabeçalho e o corpo estão preenchidos.
2	Enviar a requisição.	A API responde com status 201 (Created).
3	Ler a resposta.	O campo message mostra “Cadastro realizado com sucesso” e existe um campo _id.
4	Enviar GET /produtos/<_id recebido>.	Status 200.
5	Comparar os dados da resposta com os dados enviados.	Nome, preço, descrição e quantidade são iguais aos enviados.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-PRD-03	
ID	CT-PRD-03
Título	**Tentar cadastrar um produto com nome já existente**
Módulo / Funcionalidade	Produtos
Tipo de teste	Funcional – Negativo
Prioridade	Alta
Regra de negócio	RN-10
Pré-condições	• Preparação do ambiente feita (seção 4): o Produto A já existe. • Token do Usuário Admin válido.
Dados de teste	Nome igual ao do Produto A, preço 999, descrição “Nome repetido”, quantidade 1.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /produtos com o token do Admin e o corpo abaixo, usando o nome do Produto A. `{ "nome": "<nome do Produto A>", "preco": 999, "descricao": "Nome repetido", "quantidade": 1 }`	O cabeçalho e o corpo estão preenchidos.
2	Enviar a requisição.	A API responde com status 400 (Bad Request).
3	Ler o campo message da resposta.	Mostra “Já existe produto com esse nome”.
4	Enviar GET /produtos?nome=<nome do Produto A>.	A resposta mostra quantidade 1 e o preço continua 250: nada foi duplicado nem alterado.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 1 e PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-PRD-04	
ID	CT-PRD-04
Título	**Tentar cadastrar um produto sem token**
Módulo / Funcionalidade	Produtos
Tipo de teste	Autenticação e permissão
Prioridade	Alta
Regra de negócio	RN-04, RN-09
Pré-condições	• Nenhuma.
Dados de teste	Produto: nome “Produto Sem Token QA”, preço 100, descrição “Teste”, quantidade 5.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /produtos com o corpo abaixo, sem o cabeçalho Authorization. `{ "nome": "Produto Sem Token QA", "preco": 100, "descricao": "Teste", "quantidade": 5 }`	Há só o corpo, sem token.
2	Enviar a requisição.	A API responde com status 401 (Unauthorized).
3	Ler o campo message da resposta.	Mostra “Token de acesso ausente, inválido, expirado ou usuário do token não existe mais”.
4	Enviar GET /produtos?nome=Produto Sem Token QA.	A resposta mostra quantidade 0: o produto não foi criado.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-PRD-05	
ID	CT-PRD-05
Título	**Tentar cadastrar um produto com usuário que não é administrador**
Módulo / Funcionalidade	Produtos
Tipo de teste	Autenticação e permissão
Prioridade	Alta
Regra de negócio	RN-09
Pré-condições	• Preparação do ambiente feita (seção 4). • Token do Usuário Comum 1 válido (ele vale 10 minutos).
Dados de teste	Produto: nome “Produto Não Admin QA”, preço 100, descrição “Teste”, quantidade 5.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /produtos com o cabeçalho Authorization (token do Usuário Comum 1) e o corpo abaixo. `{ "nome": "Produto Não Admin QA", "preco": 100, "descricao": "Teste", "quantidade": 5 }`	O cabeçalho e o corpo estão preenchidos.
2	Enviar a requisição.	A API responde com status 403 (Forbidden).
3	Ler o campo message da resposta.	Mostra “Rota exclusiva para administradores”.
4	Enviar GET /produtos?nome=Produto Não Admin QA.	A resposta mostra quantidade 0: o produto não foi criado.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 2 e PRINT 3. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-PRD-06	
ID	CT-PRD-06
Título	**Editar um produto como administrador**
Módulo / Funcionalidade	Produtos
Tipo de teste	Funcional – Positivo
Prioridade	Média
Regra de negócio	RN-09
Pré-condições	• Preparação do ambiente feita (seção 4). • Token do Usuário Admin válido.
Dados de teste	Produto de teste: nome “Cabo QA <código>”, preço 15, descrição “Cabo USB”, quantidade 40. Depois da edição: preço 20 e descrição “Cabo USB-C”.

Passos de execução

Passo	Ação	Resultado esperado
1	Cadastrar o produto de teste com POST /produtos (token do Admin) e o corpo abaixo. Anotar o _id recebido. `{ "nome": "Cabo QA <código>", "preco": 15, "descricao": "Cabo USB", "quantidade": 40 }`	Status 201 e o _id do produto foi anotado.
2	Montar uma requisição PUT para /produtos/<_id> com o token do Admin e o corpo abaixo (mesmo nome e mesma quantidade; só preço e descrição mudam). `{ "nome": "Cabo QA <código>", "preco": 20, "descricao": "Cabo USB-C", "quantidade": 40 }`	O cabeçalho e o corpo estão preenchidos.
3	Enviar a requisição.	A API responde com status 200 (OK).
4	Ler o campo message da resposta.	Mostra “Registro alterado com sucesso”.
5	Enviar GET /produtos/<_id>.	O preço agora é 20 e a descrição é “Cabo USB-C”; nome e quantidade não mudaram.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 3. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-PRD-07	
ID	CT-PRD-07
Título	**Excluir um produto que não está em nenhum carrinho**
Módulo / Funcionalidade	Produtos
Tipo de teste	Funcional – Positivo
Prioridade	Média
Regra de negócio	RN-11
Pré-condições	• Preparação do ambiente feita (seção 4). • Token do Usuário Admin válido.
Dados de teste	Produto descartável: nome “Descartável QA <código>”, preço 10, descrição “Teste de exclusão”, quantidade 5.

Passos de execução

Passo	Ação	Resultado esperado
1	Cadastrar o produto descartável com POST /produtos (token do Admin) e o corpo abaixo. Anotar o _id recebido. `{ "nome": "Descartável QA <código>", "preco": 10, "descricao": "Teste de exclusão", "quantidade": 5 }`	Status 201 e o _id do produto foi anotado.
2	Enviar DELETE /produtos/<_id> com o token do Admin.	A API responde com status 200 (OK).
3	Ler o campo message da resposta.	Mostra “Registro excluído com sucesso”.
4	Enviar GET /produtos/<_id>.	Status 400 e a mensagem “Produto não encontrado”: o produto não existe mais.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 3 e PRINT 4. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-PRD-08	
ID	CT-PRD-08
Título	**Tentar excluir um produto que está dentro de um carrinho**
Módulo / Funcionalidade	Produtos
Tipo de teste	Regra de negócio
Prioridade	Alta
Regra de negócio	RN-11
Pré-condições	• Preparação do ambiente feita (seção 4): o Produto C já está no carrinho do Usuário Comum 7. • Token do Usuário Admin válido.
Dados de teste	_id do Produto C.

Passos de execução

Passo	Ação	Resultado esperado
1	Anotar o _id do Produto C.	Você tem o _id em mãos.
2	Enviar DELETE /produtos/<_id do Produto C> com o token do Admin.	A API responde com status 400 (Bad Request).
3	Ler a resposta.	O campo message mostra “Não é permitido excluir produto que faz parte de carrinho” e vem também a lista idCarrinho.
4	Enviar GET /produtos/<_id do Produto C>.	Status 200: o produto continua cadastrado.

Resultado obtido	Status 400, mensagem “Não é permitido excluir produto que faz parte de carrinho” e o produto continua cadastrado. Divergência: a lista de carrinhos veio no campo idCarrinhos (plural); o Swagger documenta idCarrinho. Resposta: {"message": "Não é permitido excluir produto que faz parte de carrinho", "idCarrinhos": ["IfuiEF5AvZX4cqvo"]}.
Status	**Falhou**
Evidência (print/exportação)	Pasta RUN/03 - Produtos: PRINT 4 (detalhe da falha). Visão geral: PRINT (GRID).
Bug relacionado	BUG-001
Observações	A regra de negócio funciona; falhou só a conferência do nome do campo, que segue o Swagger.
Executado por / Data	Yan Vieira / 21/10/2026

Grupo D — Carrinhos

Caso de teste CT-CAR-01	
ID	CT-CAR-01
Título	**Cadastrar um carrinho com dados válidos**
Módulo / Funcionalidade	Carrinhos
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-12
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 1 ainda não tem carrinho. • Token do Usuário Comum 1 válido (ele vale 10 minutos).
Dados de teste	Usuário Comum 1 e Produto A (preço 250), quantidade 2.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /carrinhos com o token do Usuário Comum 1 e o corpo abaixo. `{ "produtos": [ { "idProduto": "<_id do Produto A>", "quantidade": 2 } ] }`	O cabeçalho e o corpo estão preenchidos.
2	Enviar a requisição.	A API responde com status 201 (Created).
3	Ler a resposta.	O campo message mostra “Cadastro realizado com sucesso” e existe um campo _id (o do carrinho).
4	Enviar GET /carrinhos/<_id do carrinho>.	Status 200.
5	Conferir os dados do carrinho.	O produto é o Produto A, com quantidade 2 e precoUnitario 250; precoTotal é 500; quantidadeTotal é 2; idUsuario é o _id do Usuário Comum 1.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-CAR-02	
ID	CT-CAR-02
Título	**Reduzir o estoque do produto ao cadastrar o carrinho**
Módulo / Funcionalidade	Carrinhos
Tipo de teste	Regra de negócio
Prioridade	Alta
Regra de negócio	RN-13
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 2 ainda não tem carrinho. • Token do Usuário Comum 2 válido.
Dados de teste	Usuário Comum 2 e Produto B, quantidade 3. A letra X representa o estoque do Produto B antes do carrinho.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /produtos/<_id do Produto B> e anotar a quantidade atual (X).	Status 200 e o estoque X está anotado.
2	Montar uma requisição POST para /carrinhos com o token do Usuário Comum 2 e o corpo abaixo. `{ "produtos": [ { "idProduto": "<_id do Produto B>", "quantidade": 3 } ] }`	O cabeçalho e o corpo estão preenchidos.
3	Enviar a requisição.	A API responde com status 201 (Created).
4	Enviar GET /produtos/<_id do Produto B> de novo.	Status 200.
5	Comparar a nova quantidade com X.	A nova quantidade é X menos 3.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 1. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-CAR-03	
ID	CT-CAR-03
Título	**Tentar cadastrar um carrinho sem token**
Módulo / Funcionalidade	Carrinhos
Tipo de teste	Autenticação e permissão
Prioridade	Alta
Regra de negócio	RN-04
Pré-condições	• Nenhuma.
Dados de teste	Produto A, quantidade 1.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /carrinhos com o corpo abaixo, sem o cabeçalho Authorization. `{ "produtos": [ { "idProduto": "<_id do Produto A>", "quantidade": 1 } ] }`	Há só o corpo, sem token.
2	Enviar a requisição.	A API responde com status 401 (Unauthorized).
3	Ler o campo message da resposta.	Mostra “Token de acesso ausente, inválido, expirado ou usuário do token não existe mais”.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 1 e PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-CAR-04	
ID	CT-CAR-04
Título	**Tentar cadastrar um segundo carrinho para o mesmo usuário**
Módulo / Funcionalidade	Carrinhos
Tipo de teste	Regra de negócio
Prioridade	Alta
Regra de negócio	RN-12
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 3 ainda não tem carrinho. • Token do Usuário Comum 3 válido.
Dados de teste	Usuário Comum 3. Primeiro carrinho: Produto A, quantidade 1. Segundo carrinho: Produto B, quantidade 1.

Passos de execução

Passo	Ação	Resultado esperado
1	Cadastrar o primeiro carrinho com POST /carrinhos (token do Usuário Comum 3) e o corpo abaixo. `{ "produtos": [ { "idProduto": "<_id do Produto A>", "quantidade": 1 } ] }`	Status 201: o carrinho foi criado.
2	Montar uma segunda requisição POST para /carrinhos, com o mesmo token, usando o Produto B. `{ "produtos": [ { "idProduto": "<_id do Produto B>", "quantidade": 1 } ] }`	O cabeçalho e o corpo estão preenchidos.
3	Enviar a segunda requisição.	A API responde com status 400 (Bad Request).
4	Ler o campo message da resposta.	Mostra “Não é permitido ter mais de 1 carrinho”.
5	Enviar GET /carrinhos?idUsuario=<_id do Comum 3>.	A resposta mostra quantidade 1: só existe o primeiro carrinho.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 2. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-CAR-05	
ID	CT-CAR-05
Título	**Tentar cadastrar um carrinho com um produto que não existe**
Módulo / Funcionalidade	Carrinhos
Tipo de teste	Funcional – Negativo
Prioridade	Média
Regra de negócio	RN-12
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 4 ainda não tem carrinho. • Token do Usuário Comum 4 válido.
Dados de teste	Usuário Comum 4 e um produto com _id inventado “naoExiste1234567”, quantidade 1.

Passos de execução

Passo	Ação	Resultado esperado
1	Montar uma requisição POST para /carrinhos com o token do Usuário Comum 4 e o corpo abaixo. `{ "produtos": [ { "idProduto": "naoExiste1234567", "quantidade": 1 } ] }`	O cabeçalho e o corpo estão preenchidos.
2	Enviar a requisição.	A API responde com status 400 (Bad Request).
3	Ler o campo message da resposta.	Mostra “Produto não encontrado”.
4	Enviar GET /carrinhos?idUsuario=<_id do Comum 4>.	A resposta mostra quantidade 0: nenhum carrinho foi criado.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 2 e PRINT 3. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-CAR-06	
ID	CT-CAR-06
Título	**Tentar cadastrar um carrinho com quantidade maior que o estoque**
Módulo / Funcionalidade	Carrinhos
Tipo de teste	Regra de negócio
Prioridade	Alta
Regra de negócio	RN-12, RN-13
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 4 ainda não tem carrinho. • Token do Usuário Comum 4 válido.
Dados de teste	Usuário Comum 4 e Produto B. A letra X representa o estoque atual do Produto B; a quantidade pedida é X mais 1.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /produtos/<_id do Produto B> e anotar a quantidade atual (X).	Status 200 e o estoque X está anotado.
2	Montar uma requisição POST para /carrinhos com o token do Usuário Comum 4, pedindo X + 1 unidades (exemplo com X = 50). `{ "produtos": [ { "idProduto": "<_id do Produto B>", "quantidade": 51 } ] }`	O cabeçalho e o corpo estão preenchidos.
3	Enviar a requisição.	A API responde com status 400 (Bad Request).
4	Ler o campo message da resposta.	Mostra “Produto não possui quantidade suficiente”.
5	Enviar GET /produtos/<_id do Produto B> de novo.	A quantidade continua sendo X: o estoque não mudou.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 3. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-CAR-07	
ID	CT-CAR-07
Título	**Concluir a compra e excluir o carrinho**
Módulo / Funcionalidade	Carrinhos – concluir compra
Tipo de teste	Funcional – Positivo
Prioridade	Alta
Regra de negócio	RN-13
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 5 ainda não tem carrinho. • Token do Usuário Comum 5 válido.
Dados de teste	Usuário Comum 5 e Produto B, quantidade 1. A letra X representa o estoque do Produto B antes do carrinho.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /produtos/<_id do Produto B> e anotar a quantidade atual (X).	Status 200 e o estoque X está anotado.
2	Cadastrar o carrinho com POST /carrinhos (token do Usuário Comum 5) e o corpo abaixo. Anotar o _id do carrinho. `{ "produtos": [ { "idProduto": "<_id do Produto B>", "quantidade": 1 } ] }`	Status 201 e o _id do carrinho foi anotado.
3	Enviar DELETE /carrinhos/concluir-compra com o token do Usuário Comum 5.	A API responde com status 200 e a mensagem “Registro excluído com sucesso”.
4	Enviar GET /carrinhos/<_id do carrinho>.	Status 400 e a mensagem “Carrinho não encontrado”.
5	Enviar GET /produtos/<_id do Produto B>.	A quantidade é X menos 1: concluir a compra não devolve o estoque.

Resultado obtido	Conforme esperado.
Status	**Passou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 3 e PRINT 4. Visão geral: PRINT (GRID).
Bug relacionado	—
Observações	—
Executado por / Data	Yan Vieira / 21/10/2026

Caso de teste CT-CAR-08	
ID	CT-CAR-08
Título	**Cancelar a compra e devolver os produtos ao estoque**
Módulo / Funcionalidade	Carrinhos – cancelar compra
Tipo de teste	Regra de negócio
Prioridade	Alta
Regra de negócio	RN-13
Pré-condições	• Preparação do ambiente feita (seção 4): o Usuário Comum 6 ainda não tem carrinho. • Token do Usuário Comum 6 válido.
Dados de teste	Usuário Comum 6 e Produto A, quantidade 2. A letra X representa o estoque do Produto A antes do carrinho.

Passos de execução

Passo	Ação	Resultado esperado
1	Enviar GET /produtos/<_id do Produto A> e anotar a quantidade atual (X).	Status 200 e o estoque X está anotado.
2	Cadastrar o carrinho com POST /carrinhos (token do Usuário Comum 6) e o corpo abaixo. Anotar o _id do carrinho. `{ "produtos": [ { "idProduto": "<_id do Produto A>", "quantidade": 2 } ] }`	Status 201 e o _id do carrinho foi anotado.
3	Enviar GET /produtos/<_id do Produto A>.	A quantidade é X menos 2.
4	Enviar DELETE /carrinhos/cancelar-compra com o token do Usuário Comum 6.	A API responde com status 200 e a mensagem “Registro excluído com sucesso”.
5	Enviar GET /carrinhos/<_id do carrinho>.	Status 400 e a mensagem “Carrinho não encontrado”.
6	Enviar GET /produtos/<_id do Produto A> mais uma vez.	A quantidade voltou a ser X: o estoque foi devolvido.

Resultado obtido	Carrinho excluído e estoque devolvido (X menos 2 com o carrinho; X de novo depois do cancelamento). Divergência: a mensagem foi “Registro excluído com sucesso. Estoque dos produtos reabastecido”; o Swagger documenta “Registro excluído com sucesso”.
Status	**Falhou**
Evidência (print/exportação)	Pasta RUN/04 - Carrinhos: PRINT 4 (detalhe da falha). Visão geral: PRINT (GRID).
Bug relacionado	BUG-002
Observações	A regra de negócio funciona; falhou só a conferência do texto da mensagem, que segue o Swagger.

Executado por / Data	Yan Vieira / 21/10/2026

7. Bugs encontrados

Dos 32 casos executados, 30 passaram e 2 falharam. As duas falhas são divergências entre a documentação (Swagger) e a resposta real da API. A regra de negócio funcionou nos dois casos, por isso a severidade é Baixa. Os detalhes de cada bug estão no registro de bugs (documento à parte).

ID	Título	Caso	Severidade	Status
BUG-001	Ao excluir um produto que está em carrinho, a resposta traz o campo idCarrinhos, mas o Swagger documenta idCarrinho	CT-PRD-08	Baixa	Aberto
BUG-002	A mensagem do cancelamento de compra é diferente da documentada no Swagger	CT-CAR-08	Baixa	Aberto

8. Observações gerais

Pontos notados durante a execução que não são bugs, mas valem registro:

- As rotas GET /usuarios e GET /usuarios/{_id} devolvem a senha (password) em texto. Em um sistema real isso seria um risco de segurança; como a ServeRest é uma API de estudo, fica apenas como observação.
- A descrição do PUT /produtos/{_id} no Swagger fala em “usuário” onde deveria falar em “produto”.
- O Swagger do /login documenta apenas os status 200 e 401. A API também responde 400, com mensagem por campo, quando falta o e-mail, falta a senha ou o e-mail é inválido (CT-LOG-05, 06 e 07).
- O Swagger não descreve os campos obrigatórios nem as regras de validação dos corpos de requisição.
- Os IDs usados na URL precisam ter exatamente 16 caracteres alfanuméricos (RN-14), regra que não está no Swagger. O CT-USR-05 falhou na primeira execução por causa de um ID de teste com 17 caracteres e passou no reteste com o dado corrigido.